

CSIT 111: Fundamentals of Programming I

Lecturer: Jiayin Wang

Data Parallelism Activity

STEP 1: Pre-activity Quiz

Please take the pre-activity quiz to assess the pre-knowledge of the lab

STEP TWO: FlagMaker Activity

- You'll be pretending to be the computer drawing the flag of **Mauritius** (on the right).
- You'll use paper and crayons, but otherwise follow how a computer might draw it.
- You must fill in the squares.
- You must **color one square at a time**.
Do not color in multiple squares at one time.



Img from: <https://en.wikipedia.org/wiki/Mauritius>

FlagMaker Activity

You must **color one square at a time**.

Do not color in multiple squares at one

CORRECT!



WRONG!

Fill in the squares

WRONG!
Don't tear the
paper!



Scenario 1: A Single Processor

Two people:

1. **The Processor**: colors first stripe (cell by cell), second stripe (cell by cell), third stripe (cell by cell), and then fourth stripe (cell by cell)

2. **Timer**: Times the Processor

P1	1	2	3	4	5	6	7	8
	9	10	11	12	13	14	15	16
	17	18	19	20	21	22	23	24
	25	26	27	28	29	30	31	32
	33	34	35	36	37	38	39	40
	41	42	43	44	45	46	47	48
	49	50	51	52	53	54	55	56
	57	58	59	60	61	62	63	64

Remember: squares must be mostly colored in & you can only color one square at a time!

Scenario 1: A Single Processor

QUESTIONS

- 1. Why does this process take a long time with only one “processor”?**
- 2. How could this approach cause inefficiency in more complex tasks?**

Why does this process take a long time with only one “processor”?

- **Sequential Processing:** Each action is executed one at a time, meaning every cell is processed sequentially.
- **Single-Threaded Limitation:**
Tasks cannot be divided or handled in parallel, leading to longer completion times.
- **Processing Bottlenecks:**
This highlights how a single-threaded system becomes a bottleneck when handling multiple or repetitive tasks.

How Could This Approach Cause Inefficiency in More Complex Tasks?

- **Slow Performance for Complex Tasks:**
 - A single processor processes each item individually, making it inefficient for data-heavy tasks.
- **Scalability Issues:**
 - In general, a problem is scalable if it can handle ever increasing problem sizes.
 - While sequential processing ensures accuracy, it's not scalable for tasks requiring high efficiency or speed.
- **Inefficient Resource Utilization:**
 - Single-threaded systems often cannot handle modern workloads effectively.

Scenario 2: Two Processors Collaborating

Three people:

1. **Processor 1**: Colors red and blue stripes (cell by cell)
2. **Processor 2**: Colors yellow and green stripes (cell by cell)
3. **Timer**: Times them; stop when **all** cells are colored.

P1	1	2	3	4	5	6	7	8
	9	10	11	12	13	14	15	16
	17	18	19	20	21	22	23	24
	25	26	27	28	29	30	31	32
P2	1	2	3	4	5	6	7	8
	9	10	11	12	13	14	15	16
	17	18	19	20	21	22	23	24
	25	26	27	28	29	30	31	32

Remember: squares must be mostly colored in & you can only color one square at a time!

Scenario 2: Two Processors Collaborating

QUESTIONS

- 1. How does the addition of a second processor impact the overall completion time?**
- 2. What potential problems could arise if both processors were given overlapping tasks such as coloring the same stripes?**

How Does the Addition of a Second Processor Impact Completion Time?

- **Parallel Processing Advantage:**
With two processors, tasks can be split and executed simultaneously, e.g., one processor handles red/blue stripes, and another handles yellow/green.
- **Increased Efficiency:**
The total completion time is effectively halved, as the workload is distributed across two processors.
- **Demonstration of Scalability:**
Adding processors improves performance for divisible tasks, showing how multi-threading can optimize workflow.

What Potential Problems Could Arise if Tasks Overlap?

- **Race Conditions:**

If both processors attempt to work on the same stripes, they may overwrite each other's results, causing inconsistencies.

- **Need for Coordination:**

Proper task assignment or locking mechanisms are required to prevent overlapping and ensure accurate results.

Scenario 3: Four Processors Collaborating

Five people:

1. **Processors 1-4**: Each assigned to one color, which they color (cell by cell)
2. **Timer**: Times them; stop when **all** cells are colored.

P1	1	2	3	4	5	6	7	8
	9	10	11	12	13	14	15	16
P2	1	2	3	4	5	6	7	8
	9	10	11	12	13	14	15	16
P3	1	2	3	4	5	6	7	8
	9	10	11	12	13	14	15	16
P4	1	2	3	4	5	6	7	8
	9	10	11	12	13	14	15	16

Remember: squares must be mostly colored in & you can only color one square at a time!

Scenario 3: Four Processors Collaborating

QUESTIONS

- 1. How does using four processors further improve efficiency, and what challenges might this bring?**
- 2. What would happen if one processor finished its task significantly earlier than the others?**

How Does Using Four Processors Improve Efficiency?

- **Parallel Processing at Scale:**

- Each processor works on a single color stripe independently, drastically reducing the total completion time.
- More processors mean the task is divided into smaller portions, making processing faster and more efficient.

- **Speedup**

- Number of processors: p
- *Serial run-time* = T_{serial}
- *Parallel run-time* = T_{parallel}
- $T_{\text{parallel}} = T_{\text{serial}} / p$
- ***Speedup*** = $T_{\text{serial}} / T_{\text{parallel}}$

How Does Using Four Processors Improve Efficiency?

- **Parallel Processing at Scale:**
 - Each processor works on a single color stripe independently, drastically reducing the total completion time.
- **Scalability:**
 - More processors mean the task is divided into smaller portions, making processing faster and more efficient.
- **Challenges of Synchronization:**
 - Synchronization imposes order constraints and is used to protect access to shared data.
 - As the number of processors increases, synchronization becomes crucial to ensure they don't interfere with each other's work, which can lead to increased **coordination complexity**.

What Happens if One Processor Finishes Early?

- **Underutilization:**

A processor completing its task early remains idle while others finish their tasks, wasting resources.

- **Importance of Load Balancing:**

Distributing tasks of equal duration among processors ensures all are fully utilized and efficient.

- **Real-World Insight:**

Load balancing techniques are critical in multi-processor systems to prevent idle time and maximize throughput.

Scenario 4: Four Processors Collaborating a Different Way (Column Assignments)

Five people:

- Processors 1-4**: Each assigned a strip of two columns (first takes left 2 columns, second takes next two columns, etc.)
Processors color these the correct color cell by cell.
- Timer**: Times them; stop when **all** cells are colored.

P1		P2		P3		P4	
1	9	1	9	1	9	1	9
2	10	2	10	2	10	2	10
3	11	3	11	3	11	3	11
4	12	4	12	4	12	4	12
5	13	5	13	5	13	5	13
6	14	6	14	6	14	6	14
7	15	7	15	7	15	7	15
8	16	8	16	8	16	8	16

Remember: **squares must be mostly colored in & you can only color one square at a time!**

Scenario 4: Four Processors Collaborating a Different Way (Column Assignments)

QUESTIONS

- 1. How does assigning processors by columns rather than stripes impact performance and potential errors?**
- 2. How does this scenario reflect real-world issues in distributed systems?**

How Does Assigning Processors by Columns Impact Performance?

- **Efficiency in Task Distribution:**
Assigning processors by columns allows each to handle portions across all colors, potentially speeding up tasks when coloring large sections by stripe is inefficient.
- **Challenges at Boundaries:**
Processors risk overlapping at the edges of their assigned columns, leading to potential errors or race conditions.
- **Error Mitigation:**
Clear separation of workspaces or boundary management is essential to avoid stepping into other processors' tasks.

Real-World Reflection: Distributed Systems

- **Pipelining**

- Pipelining is a technique used in computer systems to improve efficiency by organizing tasks into sequential stages. Stages in Scenario 4: coloring Red->Blue->Yellow->Green
- Processors pass work between stages, maintain steady progress (e.g. marker sharing). Pipelining benefits systems by keeping all components active once initiated.

- **Contention and Dependencies**

- Contention occurs when processors compete for shared resources (e.g., markers in this activity)
- Dependencies: Certain tasks require others to finish before proceeding (e.g., coloring red before blue)
- Adding resources (e.g. more markers) can reduce contention but not eliminate dependencies.

Real-World Reflection: Distributed Systems

- **Spatial Task Allocation:**
Distributed systems often divide tasks spatially, similar to assigning processors to columns.
- **Boundary Management:**
Overlaps or conflicts at task boundaries mimic real-world issues like race conditions in distributed systems.
- **Coordination Protocols:**
Robust systems implement protocols to synchronize operations and ensure data integrity, just as processors need coordination here.

Animation

- https://raw.githubusercontent.com/xiaoyuansuo51-webster/Flag_Coloring_Activity/refs/heads/main/flags.html
- https://raw.githubusercontent.com/xiaoyuansuo51-webster/Flag_Coloring_Activity/refs/heads/main/parallel_linked_list.html

STEP 3: Post-Activity Quiz

Please take the post-activity quiz to assess the knowledge of the lab.

STEP 4: Post-Activity Survey